

Constraints by Construction

Lecture 5B · Parameterization is part of model design

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

A neural network can output values the model cannot use

Neural-network output

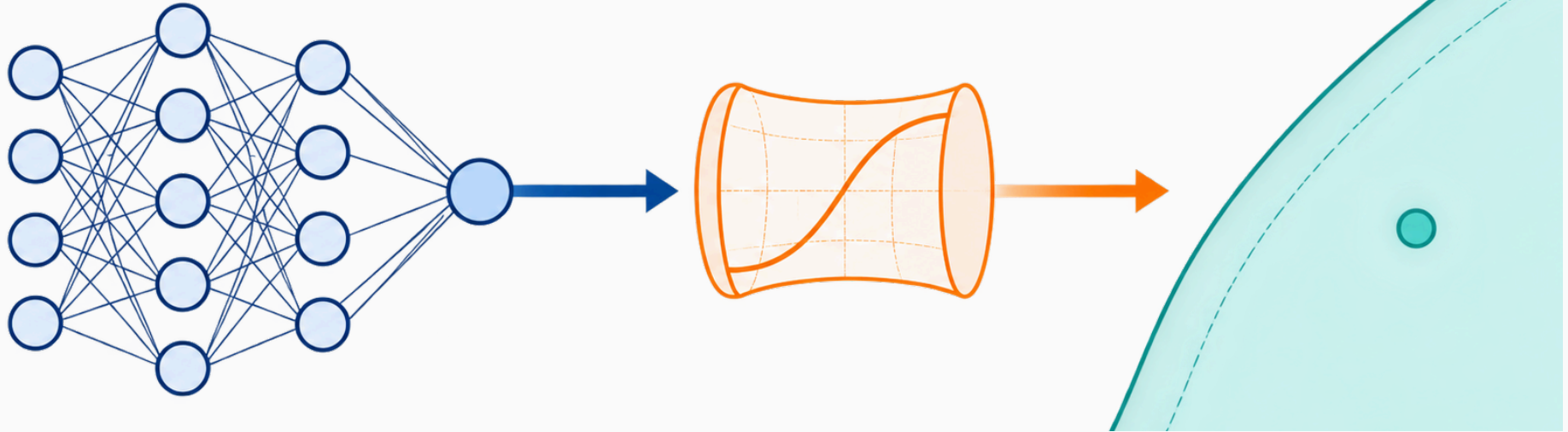
$$z = f_{w(x)} \in \mathbb{R}^d$$

Smooth map

$$\varphi : \mathbb{R}^d \rightarrow \mathcal{C}$$

Valid model parameter

$$\theta = \varphi(z) \in \mathcal{C}$$



The network may emit any $z \in \mathbb{R}^d$; φ ensures the model only sees a valid $\theta \in \mathcal{C}$.

Before optimizing: what does scale do?

For one Gaussian observation, let $r = y - \mu$ and work inside the valid domain $\sigma > 0$:

$$y \mid x \sim \mathcal{N}(\mu, \sigma^2), \quad \mathcal{L} = \underbrace{\frac{r^2}{2\sigma^2}}_{\text{fit the residual}} + \underbrace{\log \sigma}_{\text{pay for uncertainty}} + C.$$

$\sigma < 0$

Not a valid standard deviation. The probabilistic model is undefined.

$\sigma \rightarrow 0$

If $r \neq 0$, the residual term explodes. Tiny scales demand tiny errors.

σ is large

The $\log \sigma$ term grows. The model cannot inflate uncertainty for free.

What will one gradient step produce?

QUESTION

Suppose a Gaussian model learns its scale directly. At one update,

$$\sigma_0 = 0.1, \quad \frac{\partial \mathcal{L}}{\partial \sigma} = 5, \quad \eta = 0.1.$$

What is $\sigma_1 = \sigma_0 - \eta \frac{\partial \mathcal{L}}{\partial \sigma}$?

A. 0.6: the step increases uncertainty

B. 0.4: take the absolute value

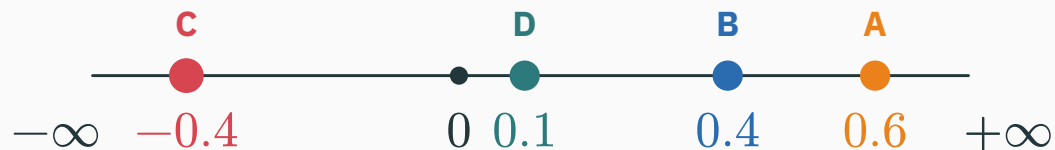
C. -0.4: the step crosses zero

D. 0.1: the model blocks invalid moves

A gradient step can make σ invalid

A ANSWER

$$\sigma_1 = 0.1 - 0.1(5) = -0.4.$$



Correct: C — -0.4

Why: Gradient descent follows the local slope, even when the step makes σ invalid.

What failed?

The update followed the usual gradient-descent rule:

$$\text{new} = \text{old} - \eta \times \text{gradient}.$$

The problem is the coordinate: σ is valid only on $(0, \infty)$, but direct optimization treats it as an unconstrained real number.

Correct fix: make invalid values unreachable

**Positive** $s \in \mathbb{R} \rightarrow \sigma = e^s$ or $\sigma = \text{softplus}(s) + \varepsilon, \varepsilon > 0$; either gives $\sigma > 0$ **Probability** $z \in \mathbb{R} \rightarrow p = \text{sigmoid}(z) \in (0, 1)$ **Probability vector (simplex)** $\mathbf{p} = \text{softmax}(\mathbf{z}) : p_k > 0, \sum_k p_k = 1$ **Positive-definite covariance**raw vector $\rightarrow$ lower-triangular factor $\rightarrow \Sigma \succ 0$

$z \in \mathbb{R}^d$: unrestricted network output; d is its length. φ (**phi**): constraint map.

$\mathcal{C}$: allowed parameter set. $\theta = \varphi(z) \in \mathcal{C}$: valid parameter used by the model.

Softplus smoothly turns any real number positive

Take any real-valued raw coordinate u and define

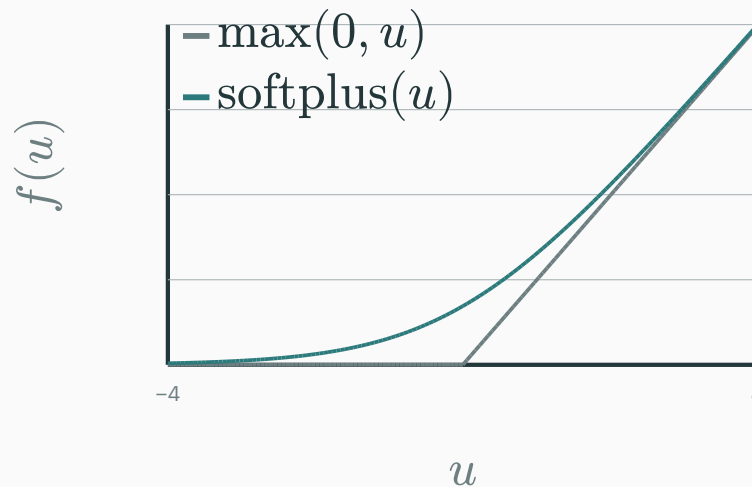
$$\text{softplus}(u) = \log(1 + e^u).$$

Constraint guarantee

$$\sigma = \text{softplus}(u) + \varepsilon > \varepsilon \text{ for every finite } u.$$

Learning signal

Softplus is increasing, and its slope stays below 1.



Softplus rounds off the ReLU corner while remaining strictly above zero.

$$\begin{aligned} u = -4 &\rightarrow 0.018 & u = 0 &\rightarrow \log 2 \approx 0.693 \\ u = 3 &\rightarrow 3.05 \end{aligned}$$

Can you differentiate softplus?

Let $a(u) = 1 + e^u$. Then $\text{softplus}(u) = \log(a(u))$.

Can you differentiate softplus?

QUESTION

Let $a(u) = 1 + e^u$. Then $\text{softplus}(u) = \log(a(u))$.

1. Inner derivative

$$a'(u) = 0 + e^u = e^u.$$

2. Outer derivative

$$\frac{\partial}{\partial a} \log a = \frac{1}{a}.$$

3. Apply the chain rule

$$\frac{\partial \text{softplus}(u)}{\partial u} = \frac{1}{a(u)} \times a'(u)$$

$$= \frac{1}{1 + e^u} \times e^u = \frac{e^u}{1 + e^u}.$$

The derivative of softplus is a sigmoid

Divide the numerator and denominator of $\frac{e^u}{1+e^u}$ by e^u .

$$\begin{aligned}\frac{e^u}{1+e^u} &= \frac{\frac{e^u}{e^u}}{\frac{1+e^u}{e^u}} \\ &= \frac{1}{e^{-u} + 1} = \frac{1}{1 + e^{-u}} \\ &= \text{sigmoid}(u).\end{aligned}$$

Large negative u

The slope is near 0.

$u = 0$

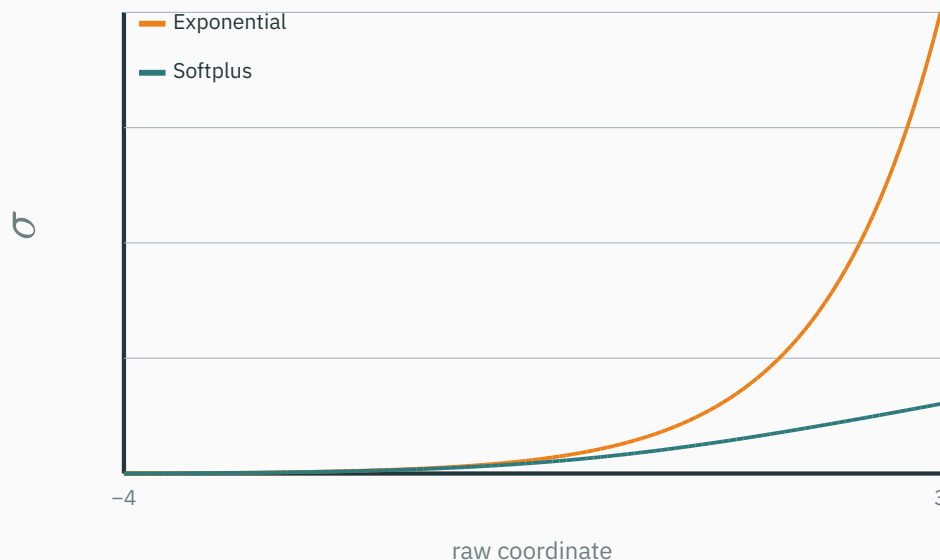
The slope is exactly $\frac{1}{2}$.

Large positive u

The slope is near 1.

For finite u in exact arithmetic, $0 < \text{sigmoid}(u) < 1$; softplus has no exactly flat branch.

The same raw step is gentler under softplus



Starting from $\sigma = 10$, take the same raw step $\Delta = +0.1$.

Exponential	$10 \rightarrow 11.05$
-------------	------------------------

Softplus	$10 \rightarrow 10.10$
----------	------------------------

Here we use softplus: large predicted scales move gently.

For Gaussian regression, we use softplus

```
mu, raw_scale = net(x).unbind(-1) # net(x): [N, 2]
assert y.ndim == 2 and y.shape[-1] == 1
y = y[:, 0] # [N, 1] -> [N]
assert y.shape == mu.shape

# Chosen here: positive scale + a small numerical floor
sigma = F.softplus(raw_scale) + 1e-4

# Also valid, but not used here:
# sigma = raw_scale.exp()

loss = -Normal(mu, sigma).log_prob(y).mean()
```

PyTorch `softplus` documentation

Which map?

We use **softplus** from here on. `exp` would also keep σ positive, but it is a different map. Pick one.

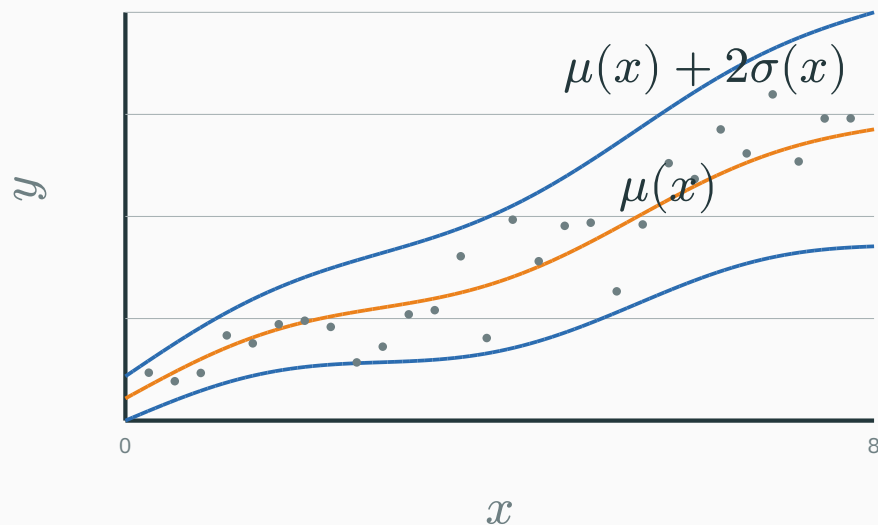
What does PyTorch handle?

`Normal.log_prob(y)` evaluates the Gaussian likelihood; autograd differentiates through `softplus`. No hand derivation is needed.

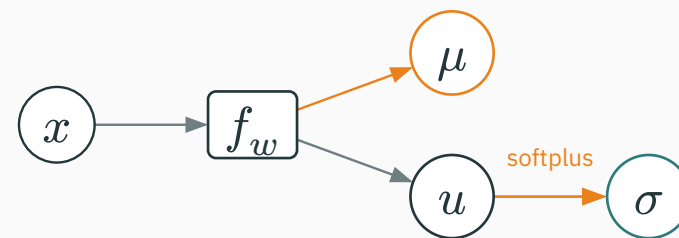
Why assert shapes?

Keep μ and y both `[N]`; otherwise broadcasting can silently produce `[N, N]` log probabilities.

Heteroskedastic regression makes the scale visible



— predicted mean $\mu(x)$ — $\mu(x) \pm 2\sigma(x)$ noise widens with x



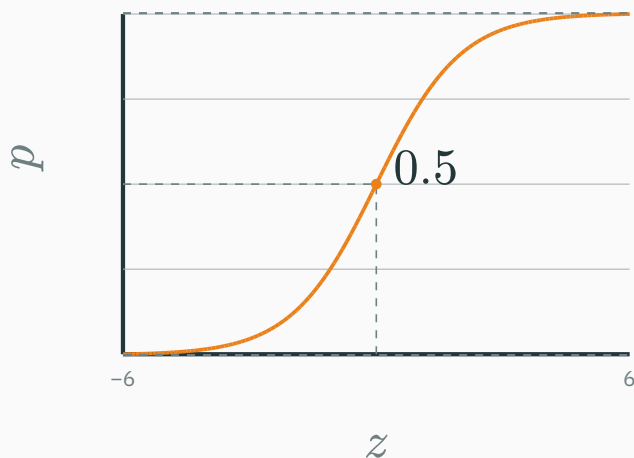
Two outputs, two meanings

$\mu(x)$ is the conditional mean. Softplus maps the raw $u(x)$ to a valid conditional scale.

Under the Gaussian model, $\sigma(x)$ is the conditional residual, or aleatoric, scale. It does not measure epistemic uncertainty about the network.

Sigmoid maps one logit to a valid probability

$$p = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}, \quad z \in \mathbb{R}.$$



In exact arithmetic, finite logits map into the open interval $(0, 1)$.

z	$p = \text{sigmoid}(z)$
-2	0.119
0	0.500
2	0.881

Negative logits favor class 0; positive logits favor class 1. Running the map backward gives the **log-odds**; let us unpack that scale next.

Finite precision can saturate sigmoid to exactly 0 or 1 for extreme logits.

Odds compare class 1 with class 0

$$\text{odds} = \frac{p}{1-p}.$$

In binary classification, p is the probability of class 1 and $1 - p$ is the probability of class 0.

$$p = 0.01$$

$$\text{odds} = \frac{0.01}{0.99}$$

$$= \frac{1}{99} \approx 0.010$$

Class 0 is strongly favored.

$$p = 0.50$$

$$\text{odds} = \frac{0.50}{0.50}$$

$$= 1$$

Neither class is favored.

$$p = 0.99$$

$$\text{odds} = \frac{0.99}{0.01}$$

$$= 99$$

Class 1 is strongly favored.

Odds < 1: class 0

Odds = 1: tie

Odds > 1: class 1

Log-odds stretch probabilities across the real line

$$z = \text{logit}(p) = \log(\text{odds}) = \log\left(\frac{p}{1-p}\right).$$

Taking the logarithm sends odds below 1 negative, odds of 1 to zero, and odds above 1 positive.

$$p = 0.01$$

$$z = \log\left(\frac{1}{99}\right)$$

$$\approx -4.60$$

$$p \rightarrow 0^+ \Rightarrow z \rightarrow -\infty$$

$$p = 0.50$$

$$z = \log(1)$$

$$= 0$$

$$p = 0.5 \Rightarrow z = 0$$

$$p = 0.99$$

$$z = \log(99)$$

$$\approx +4.60$$

$$p \rightarrow 1^- \Rightarrow z \rightarrow +\infty$$

Inverse maps: $p = \text{sigmoid}(z)$ if and only if $z = \text{logit}(p)$.

Can you rewrite BCE using $\text{softplus}(z)$?

Let $y \in \{0, 1\}$, $p = \text{sigmoid}(z)$, and $\ell(z, y) = \text{BCE}(\text{sigmoid}(z), y)$.

$$\ell(z, y) = -y \log p - (1 - y) \log(1 - p).$$

Can you rewrite BCE using $\text{softplus}(z)$?

QUESTION

Let $y \in \{0, 1\}$, $p = \text{sigmoid}(z)$, and $\ell(z, y) = \text{BCE}(\text{sigmoid}(z), y)$.

$$\ell(z, y) = -y \log p - (1 - y) \log(1 - p).$$

1. Rewrite $\log p$

$$p = \frac{e^z}{1 + e^z}$$

$$\log p = z - \log(1 + e^z)$$

2. Rewrite $\log(1 - p)$

$$1 - p = \frac{1}{1 + e^z}$$

$$\log(1 - p) = -\log(1 + e^z)$$

3. Substitute and collect the common term

$$\begin{aligned}\ell(z, y) &= -y(z - \log(1 + e^z)) + (1 - y) \log(1 + e^z) \\ &= -yz + [y + (1 - y)] \log(1 + e^z) = \text{softplus}(z) - yz.\end{aligned}$$

A correct formula can still overflow on a computer

$\text{softplus}(z) = \log(1 + e^z)$ **is mathematically correct.**

A modest input: $z = 2$

$$e^2 \approx 7.389$$

$$\log(1 + e^2) \approx 2.1269.$$

The direct computation creates no unusually large number, so it works.

A large input: $z = 1000$

$$e^{1000} \rightarrow \infty \text{ in float32}$$

$$\log(1 + \infty) = \infty.$$

But the true value is approximately 1000, not infinity. The computer overflowed before taking the logarithm.

What we need: the same mathematical function, evaluated in an order that never constructs a huge exponential.

Can we evaluate softplus without a large exponential?

QUESTION

Start from $\text{softplus}(z) = \log(1 + e^z)$ and separate the two signs of z .

Can we evaluate softplus without a large exponential?

Start from $\text{softplus}(z) = \log(1 + e^z)$ and separate the two signs of z .

If $z \geq 0$: move the large term outside

$$\begin{aligned}\log(1 + e^z) \\ &= \log(e^{z(1+e^{-z})}) \\ &= z + \log(1 + e^{-z}).\end{aligned}$$

Now $-z \leq 0$, so the exponential is at most 1.

If $z < 0$: the original form is already safe

$$\begin{aligned}0 < e^z < 1 \\ \log(1 + e^z) \\ &= 0 + \log(1 + e^{-|z|}).\end{aligned}$$

Here $-|z| = z$, so we have not changed the value.

One expression covers both cases

$$\text{softplus}(z) = \max(z, 0) + \log(1 + e^{-|z|}).$$

Because $-|z| \leq 0$, the remaining exponential satisfies $e^{-|z|} \leq 1$ and cannot overflow.

Same answer normally; stable answer at extremes

D DERIVATION

Compare float32 outputs

z	Direct formula	Stable route
-1000	0	0
2	2.1269	2.1269
1000	inf	1000

At ordinary inputs the answers agree. At $z = 1000$, only the stable evaluation returns the finite answer.

Check it in PyTorch

```
z = torch.tensor([-1000., 2., 1000.])

naive = torch.log1p(torch.exp(z))
stable = F.softplus(z)

# naive:  tensor([0., 2.1269, inf])
# stable: tensor([0., 2.1269, 1000.])
```

`log1p` improves accuracy when its input is small, but it cannot rescue an `exp(z)` that has already overflowed.

In practice: use `F.softplus`. For binary classification, use the fused BCE-with-logits operation introduced after the gradient derivation.

Is underflow a problem too?

At $z = +1000$ or $z = -1000$, floating point rounds $e^{-|z|}$ to 0.

Does that break the stable formula?

Is underflow a problem too?

QUESTION

$$z = +1000$$

$$e^{-1000} \rightarrow 0$$

$$\text{softplus}(z) = 1000 + \log(1 + 0) = 1000.$$

Only an unrepresentably tiny correction was discarded.

$$z = -1000$$

$$e^{-1000} \rightarrow 0$$

$$\text{softplus}(z) = 0 + \log(1 + 0) = 0.$$

The exact answer is positive, about e^{-1000} , but too small to store.

Forward value: overflow turned a useful finite answer into ∞ ; underflow here drops only a tiny correction. For a scale, $\sigma = \text{softplus}(u) + \varepsilon$ still gives $\sigma > \varepsilon$.

Learning caveat: $\text{softplus}'(z) = \text{sigmoid}(z)$ also rounds to 0 at $z = -1000$, so an extremely negative raw scale can learn very slowly.

What is the BCE gradient with respect to the logit?

Start from the smooth expression we just derived:

$$\ell(z, y) = \text{softplus}(z) - yz.$$

What is the BCE gradient with respect to the logit?

Start from the smooth expression we just derived:

$$\ell(z, y) = \text{softplus}(z) - yz.$$

Differentiate with respect to the logit

$$\frac{\partial \ell}{\partial z} = \text{sigmoid}(z) - y = p - y.$$

We used $\text{softplus}'(z) = \text{sigmoid}(z) = p$.

If $p - y < 0$

The loss has a **negative slope** at the current z . Moving z upward makes the loss fall.

If $p - y > 0$

The loss has a **positive slope** at the current z . Moving z downward makes the loss fall.

The crucial next step: gradient descent subtracts this slope.

Which way will the logit move?

$$\text{new } z = \text{old } z - \eta(p - y).$$

Target $y = 1$, **current probability** $p = 0.2$, **learning rate** $\eta = 0.1$.

Which way will the logit move?

$$\text{new } z = \text{old } z - \eta(p - y).$$

Target $y = 1$, **current probability** $p = 0.2$, **learning rate** $\eta = 0.1$.

One example

$$p - y = 0.2 - 1 = -0.8$$

$$\begin{aligned} \text{new } z &= \text{old } z - 0.1(-0.8) \\ &= \text{old } z + 0.08. \end{aligned}$$

Subtracting a negative slope raises z . A larger logit gives a larger probability p , moving the prediction toward the target $y = 1$.

The same $p - y$ signal trains the whole network

Now return to the real network: the optimizer updates θ . For $z = f_{\theta}(x)$, apply the chain rule:

Error signal $\times$ sensitivity of the logit

$$\nabla_{\theta} \ell = (p - y) \nabla_{\theta} z.$$

The product combines the prediction error with each parameter's effect on the logit.

For $z = w^T h + b$

$$\frac{\partial \ell}{\partial w} = (p - y) h$$

$$\frac{\partial \ell}{\partial b} = p - y$$

$$\frac{\partial \ell}{\partial h} = (p - y) w.$$

What the two factors mean

$p - y$ says whether the prediction should move **up or down**, and how strongly.

$\nabla_{\theta} z$ says how each parameter changes that prediction.

Why not form p first?

```
p = torch.sigmoid(z)
loss = -(y * p.log()
        + (1 - y) * torch.log1p(-p)).mean()
```

With $z = 100$ and $y = 0$, float32 sigmoid may round p to 1, so $-\log(1 - p) \rightarrow \infty$.

Forming p first can make the naive formula infinite.

Use the fused primitive

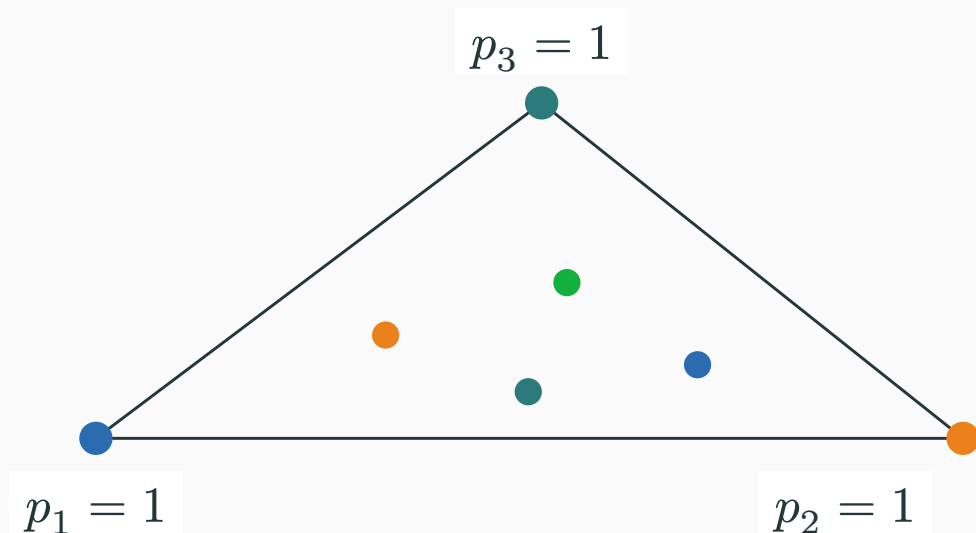
```
assert y.dtype.is_floating_point
assert y.shape == z.shape
assert ((0 <= y) & (y <= 1)).all()

loss = F.binary_cross_entropy_with_logits(z, y)
p = z.sigmoid() # interpretation/reporting only
```

For $z = 100$, $y = 0$, the fused loss is approximately 100, not infinity.

PyTorch BCEWithLogitsLoss: fused sigmoid and BCE

Softmax maps logits into the simplex



For $K = 3$, the simplex is this triangle of valid probability vectors. An edge means at least one class has probability 0.

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad z \in \mathbb{R}^K.$$

The simplex is the set of vectors with $p_k \geq 0$ and $\sum_k p_k = 1$.

In exact arithmetic, finite logits guarantee

$$p_k > 0, \quad \sum_k p_k = 1.$$

Thus softmax lands strictly **inside** the simplex, never exactly on an edge. Boundary distributions are approached as logit differences grow.

What does naive softmax compute at extreme logits?

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Try $\mathbf{z} = (1000, 999, 998)$ and $\mathbf{z} = (-1000, -1001, -1002)$ in float32.

What does naive softmax compute at extreme logits?

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Try $z = (1000, 999, 998)$ and $z = (-1000, -1001, -1002)$ in float32.

Overflow: exponentials become infinite

$$z = (1000, 999, 998)$$

$$\text{float32: } e^z \rightarrow (\infty, \infty, \infty)$$

$$p_k = \frac{\infty}{\infty} \rightarrow \text{NaN}.$$

Underflow: exponentials become zero

$$z = (-1000, -1001, -1002)$$

$$\text{float32: } e^z \rightarrow (0, 0, 0)$$

$$p_k = \frac{0}{0} \rightarrow \text{NaN}.$$

Both vectors have the same logit gaps and should give $p \approx (0.665, 0.245, 0.090)$. The probability distribution exists; the direct floating-point algorithm failed before it could normalize.

Does adding the same constant change softmax?

Let c be the same scalar added to every logit.

$$p'_k = \frac{e^{z_k+c}}{\sum_j e^{z_j+c}}.$$

Does adding the same constant change softmax?

QUESTION

Let c be the same scalar added to every logit.

$$p'_k = \frac{e^{z_k+c}}{\sum_j e^{z_j+c}}.$$

$$\begin{aligned} p'_k &= \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} \\ &= \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k. \end{aligned}$$

Only the gaps matter

$$(z_i + c) - (z_j + c) = z_i - z_j.$$

The common offset carries no class information.

Exact consequence

$$(1000, 999, 998) = (2, 1, 0) + 998(1, 1, 1).$$

These vectors have exactly the same softmax in exact arithmetic.

A small example shows shifting preserves probabilities

Quantity	Direct: $\mathbf{z} = (2, 1, 0)$	Shifted: $\mathbf{r} = (0, -1, -2)$
Logits	$(2, 1, 0)$	$(0, -1, -2)$
Exponentials	$(7.389, 2.718, 1.000)$	$(1.000, 0.368, 0.135)$
Sum	11.107	1.503
Probabilities	$(0.665, 0.245, 0.090)$	$(0.665, 0.245, 0.090)$

Why the ratios match

$$e^{z_j - 2} = \frac{e^{z_j}}{e^2}.$$

Every numerator and the denominator were divided by the same positive number e^2 , so every ratio is unchanged.

The loss is unchanged too: for class 1, $-\log(p_1) \approx 0.408$ along either route. For $\mathbf{z} = (1000, 999, 998)$, subtracting its maximum also gives $\mathbf{r} = (0, -1, -2)$; next we work that extreme case through.

The same shift rescues $z = (1000, 999, 998)$

Direct fails; shifting first works

Direct float32: $e^z \rightarrow (\infty, \infty, \infty)$, so $p \rightarrow (\text{NaN}, \text{NaN}, \text{NaN})$.

Shifted route:

1. $m = \max_j z_j = 1000$
2. $r_j = z_j - m$, so $r = (0, -1, -2)$
3. $w = e^r \approx (1, 0.368, 0.135)$
4. $S = \sum_j w_j \approx 1.503$
5. $p = \frac{w}{S} \approx (0.665, 0.245, 0.090)$.

Very small probabilities may still round to zero. That is acceptable for reporting, but a training loss should stay in log space.

Why the shifted arithmetic is safe

$r_j = z_j - m \leq 0$ for every class.

At least one $r_j = 0$.

Therefore $0 \leq e^{r_j} \leq 1$.

The denominator contains $e^0 = 1$, so

$$1 \leq \sum_j e^{r_j} \leq K.$$

No exponential overflows, and the denominator cannot become zero.

Stable cross-entropy ($m = \max_j z_j$)

$$\begin{aligned}\ell(\mathbf{z}, t) &= -\log p_t \\ &= -z_t + \log\left(\sum_j e^{z_j}\right) \\ &= -(z_t - m) + \log\left(\sum_j e^{z_j - m}\right).\end{aligned}$$

Why log space matters

For $\mathbf{z} = (0, 1000)$ and **mathematical class 1**, softmax may round to $(0, 1)$, so $-\log(p_1) = \infty$. The stable expression gives

$$1000 + \log(1 + e^{-1000}) \approx 1000.$$

Use the fused primitives

```
# logits: [N, K]
# target: [N], class indices 0,...,K-1
loss = F.cross_entropy(logits, target)

# only when these values are needed
log_probs = F.log_softmax(logits, dim=-1)
probs = logits.softmax(dim=-1)
```

`cross_entropy` accepts unnormalized logits;
`log_softmax` avoids computing
`log(softmax(...))` as two unstable operations.

PyTorch `cross_entropy` · PyTorch `log_softmax`

Stable does not make floating-point exact or repair NaN or infinite input logits.

A multivariate Gaussian needs a covariance matrix

$$y \sim \mathcal{N}(\mu, \sigma^2) \quad \rightarrow \quad \mathbf{Y} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma).$$

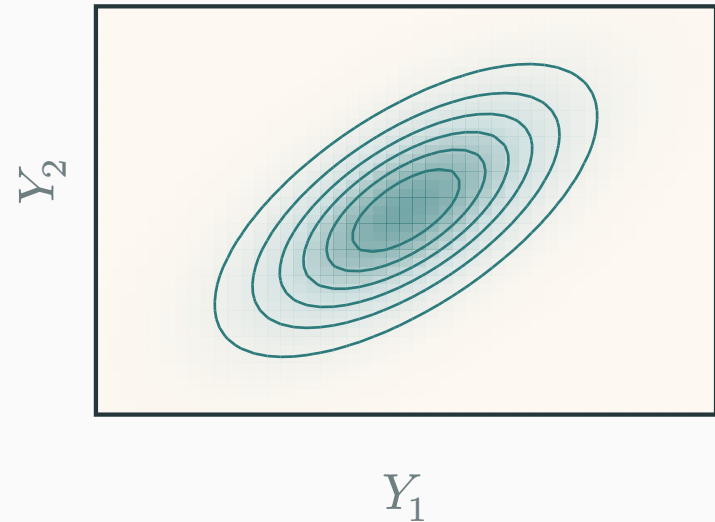
The definition

$$\boldsymbol{\mu} = \mathbb{E}[\mathbf{Y}],$$

$$\Sigma = \text{Cov}(\mathbf{Y}) = \mathbb{E}[(\mathbf{Y} - \boldsymbol{\mu})(\mathbf{Y} - \boldsymbol{\mu})^T].$$

$$\Sigma = \begin{pmatrix} \text{Var}(Y_1) & \text{Cov}(Y_1, Y_2) \\ \text{Cov}(Y_1, Y_2) & \text{Var}(Y_2) \end{pmatrix}.$$

In one dimension, variance sets the width. In several dimensions, covariance sets the widths and tilt of the Gaussian cloud.



Equal-density contours form a tilted ellipse around $\boldsymbol{\mu}$.

Read the entries

Diagonal: each output's variance.

Off-diagonal: how the two outputs move together.

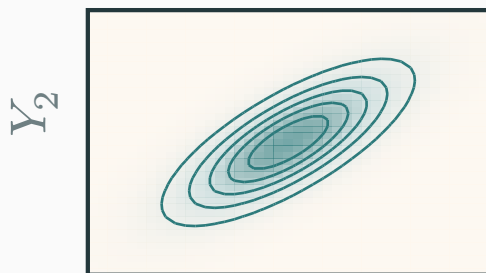
The repeated off-diagonal entry makes $\Sigma = \Sigma^T$.

Covariance shows how two outputs move together

$$\text{Cov}(Y_1, Y_2) = \mathbb{E}[(Y_1 - \mu_1)(Y_2 - \mu_2)].$$

Each observation contributes the product of its two deviations from the means.

Positive covariance

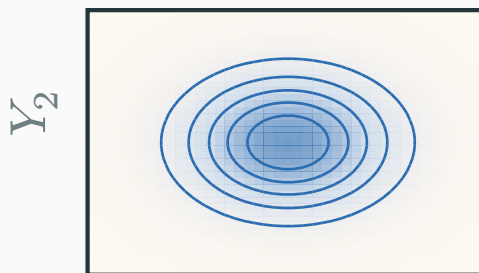


Y_1

$$\rho = 0.75$$

Same-sign deviations reinforce.

Zero covariance

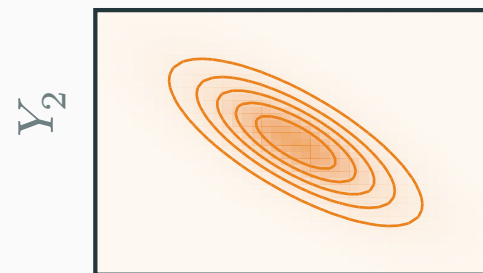


Y_1

$$\rho = 0$$

Positive and negative products cancel.

Negative covariance



Y_1

$$\rho = -0.75$$

Opposite-sign deviations dominate.

$\rho = \frac{\text{Cov}(Y_1, Y_2)}{\text{sd}(Y_1)\text{sd}(Y_2)} \in [-1, 1]$ is unitless. Covariance carries multiplied units—for example, $\text{cm} \cdot \text{kg}$. For a jointly Gaussian pair, $\rho = 0$ also means independence.

PSD vs PD: can a combination have zero variance?

Choose weights $\mathbf{w} = (w_1, w_2)$ and combine the outputs:

$$S = w_1 Y_1 + w_2 Y_2, \quad \text{Var}(S) = \mathbf{w}^T \Sigma \mathbf{w}.$$

$\mathbf{w}$ is simply the pair of weights; S is one weighted sum of the two outputs.

Positive semidefinite (PSD)

Every choice of weights gives $\text{Var}(S) \geq 0$; a nonzero choice may still give 0.

$$\Sigma = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{w} = (1, -1)$$

$$S = Y_1 - Y_2, \quad \text{Var}(S) = 0.$$

The difference is constant: the two outputs are perfectly locked together.

Positive definite (PD)

Every nonzero choice of weights gives $\text{Var}(S) > 0$; zero is not allowed.

$$\Sigma = I_2$$

$$\text{Var}(S) = w_1^2 + w_2^2 > 0 \text{ if } \mathbf{w} \neq \mathbf{0}.$$

No nontrivial weighted combination is perfectly fixed.

Why does $\Sigma = LL^T$ guarantee positive variance?

For a 2-D target, the network emits three unrestricted numbers:

$$\mathbf{q} = (q_{11}, q_{21}, q_{22}) \in \mathbb{R}^3.$$

$$L(\mathbf{q}) = \begin{pmatrix} \text{softplus}(q_{11}) + \epsilon & 0 \\ q_{21} & \text{softplus}(q_{22}) + \epsilon \end{pmatrix}$$

$$\Sigma = LL^T.$$

Read the construction

$\mathbf{q}$: raw network outputs.

L : lower-triangular factor.

Softplus: makes the two diagonal entries positive.

$\epsilon > 0$: a small numerical floor.

Why does $\Sigma = LL^T$ guarantee positive variance?

For a 2-D target, the network emits three unrestricted numbers:

$$\mathbf{q} = (q_{11}, q_{21}, q_{22}) \in \mathbb{R}^3.$$

$$L(\mathbf{q}) = \begin{pmatrix} \text{softplus}(q_{11}) + \epsilon & 0 \\ q_{21} & \text{softplus}(q_{22}) + \epsilon \end{pmatrix}$$

$$\Sigma = LL^T.$$

Read the construction

$\mathbf{q}$: raw network outputs.

L : lower-triangular factor.

Softplus: makes the two diagonal entries positive.

$\epsilon > 0$: a small numerical floor.

Pick any nonzero weights $\mathbf{w}$. Positive diagonal entries make L invertible, so $L^T \mathbf{w} \neq \mathbf{0}$ and $\mathbf{w}^T \Sigma \mathbf{w} = \|L^T \mathbf{w}\|^2 > 0$. Thus every nontrivial weighted combination has positive variance, so Σ is PD.

```
mu, q = model(x)          # [B, 2], [B, 3]

L = q.new_zeros(q.shape[:-1] + (2, 2))
L[..., 0, 0] = F.softplus(q[..., 0]) + 1e-4
L[..., 1, 0] = q[..., 1]
L[..., 1, 1] = F.softplus(q[..., 2]) + 1e-4

dist = MultivariateNormal(mu, scale_tril=L)
loss = -dist.log_prob(y).mean()
```

PyTorch MultivariateNormal documentation

`scale_tril=L` passes the factor directly; PyTorch need not form or invert $\Sigma = LL^T$.

How gradients reach q

Autograd applies the chain rule. For the first diagonal entry,

$$\frac{\partial \ell}{\partial q_{11}} = \frac{\partial \ell}{\partial L_{11}} \text{sigmoid}(q_{11}).$$

This uses $\text{softplus}'(q) = \text{sigmoid}(q)$; the same rule holds for q_{22} .

Since $L_{21} = q_{21}$, its gradient passes through unchanged.

Unlike ReLU, `softplus` has no exactly flat negative branch, although its slope can become very small for a very negative input.

One recipe: raw outputs in, valid parameters out

The network may output any real numbers. A fixed transformation gives the model only legal parameters.

Model needs	Raw output	Transformation
Positive scale	$u \in \mathbb{R}$	$\sigma = \text{softplus}(u) + \varepsilon$
Probability	$z \in \mathbb{R}$	$p = \text{sigmoid}(z)$
Probability vector	$\mathbf{z} \in \mathbb{R}^K$	$\mathbf{p} = \text{softmax}(\mathbf{z})$
PD covariance	raw lower-triangle entries	positive-diagonal L ; $\Sigma = LL^T$

Main idea: let the network learn unrestricted coordinates, and enforce the model's domain in the transformation.